

Análisis técnico del microservicio Spring Boot J48

Evaluación de arquitectura, operación, seguridad y calidad del componente de predicción de riesgo de recaída psicológica.

Alcance: `services/j48-service`

Base técnica: Spring Boot 3.4.5 · Java 25 · Weka 3.8.6 · Spring AI 1.0.0

Fecha de elaboración: 31 de agosto de 2026

Nota de uso clínico. Este componente es apoyo a la decisión, no un sistema de diagnóstico ni de prescripción. Sus resultados requieren revisión profesional y no deben tratarse como una conclusión clínica autónoma.

Resumen ejecutivo

Resultado del análisis. El repositorio implementa un microservicio Java independiente que entrena, persiste y expone un árbol de decisión J48 para estimar riesgo de recaída psicológica. Su diseño es consistente con una arquitectura por capas: API HTTP, servicios de dominio, repositorio de artefacto del modelo, DTO, configuración, seguridad y manejo centralizado de errores. El componente no usa base de datos; persiste un modelo Weka serializado en el sistema de archivos.

5

Endpoints funcionales
salud, metadatos, predicción, explicación y reentrenamiento.

3

Niveles de calidad
validación de entrada, respuesta de errores uniforme y pruebas focalizadas.

1

Artefacto persistido
modelo J48 y cabecera de atributos, serializados localmente.

0

Datos clínicos enviados por defecto
la explicación generativa está deshabilitada hasta configurar proveedor y clave.

Conclusiones principales

- La separación de responsabilidades es adecuada para un microservicio de inferencia sin persistencia relacional.
- La ruta administrativa `POST /train` admite protección con `X-J48-Admin-Token`; el perfil de producción exige activarla.
- El modelo puede autocrearse desde el ARFF al iniciar. Esto simplifica el despliegue, pero requiere control de versión y validación del conjunto de entrenamiento.
- La integración con Spring AI aplica minimización de datos y dispone de explicación local de respaldo si el proveedor no está habilitado.
- La mayor brecha es operativa: falta evidencia de ejecución de pruebas en CI para este servicio y de autenticación fuerte entre servicios.

Contenido

1. Alcance y contexto
2. Arquitectura y flujo de datos
3. Diseño por capas
4. Contratos de API
5. Modelo J48 y ciclo de vida
6. Configuración y despliegue
7. Seguridad, privacidad y cumplimiento
8. Calidad, pruebas y observabilidad
9. Riesgos y recomendaciones priorizadas
10. Conclusión y evidencia revisada

1. Alcance y contexto

El proyecto **Centro COP — Plataforma clínica escalable** es un monorepositorio que integra aplicaciones web públicas y de gestión clínica, una API principal NestJS, servicios de inteligencia artificial y componentes de infraestructura. El elemento revisado en este informe es `services/j48-service`, el microservicio Spring Boot dedicado al cálculo de riesgo de recaída.

La fuente de entrenamiento esperada es `datasets/relapse_risk_j48.arff`. El servicio lee los atributos definidos en ese archivo y usa el último atributo —configurado como clase cuando el ARFF no lo declara— para entrenar el clasificador `weka.classifiers.trees.J48`.

Elemento	Decisión observada	Implicación
Tipo de servicio	API REST sin estado de sesión	Escalable horizontalmente si el modelo se distribuye de forma controlada.
Persistencia	Archivo serializado de Weka	No necesita JPA ni entidad; requiere volumen persistente y estrategia de versionado.
IA generativa	Opcional, vía Spring AI/OpenAI	La predicción funciona sin proveedor externo; la explicación tiene fallback.
Integración	Docker Compose con perfil <code>j48-java</code>	El servicio puede activarse independientemente del stack principal.

Dependencias relevantes

Dependencia	Propósito
Spring Boot Web / Validation	API REST, deserialización JSON y validación declarativa de solicitudes.
Spring Security	Cadena de filtros y control del token administrativo para reentrenamiento.
Spring Boot Actuator	Endpoint de salud para operación y orquestación.
springdoc OpenAPI	Contrato navegable mediante Swagger UI y especificación OpenAPI.
Weka 3.8.6	Carga de ARFF, entrenamiento, clasificación y distribución de probabilidades J48.
Spring AI	Explicación clínica opcional, desacoplada de la inferencia del modelo.

2. Arquitectura y flujo de datos

La arquitectura mantiene el controlador ligero: recibe y valida la solicitud, delega el comportamiento a servicios de dominio y devuelve DTO. El acceso al artefacto del modelo está aislado en un repositorio, evitando que la lógica de serialización se mezcle con la inferencia.



Flujo de inicio

1. Al iniciar, `J48ModelService.init()` busca el modelo configurado.
2. Si existe, carga el clasificador y su cabecera de atributos desde el archivo serializado.
3. Si no existe y `J48_AUTO_TRAIN=true`, entrena desde el ARFF y persiste el resultado.
4. Si no existe y el autoentrenamiento está deshabilitado, detiene el arranque con un error explícito.

Flujo de predicción

1. Se rechaza una solicitud sin características.
2. Para cada atributo del modelo se busca un valor por nombre en `features`.
3. Valores ausentes, no numéricos o nominales no reconocidos se marcan como faltantes; Weka procesa el caso según el árbol entrenado.
4. Se responde la clase predicha, su índice y la distribución de probabilidades por etiqueta.

Implicación: la API es tolerante a campos extra y a atributos omitidos. Es conveniente que el consumidor valide el esquema clínico antes de invocar el servicio para no degradar la calidad de la inferencia con valores faltantes.

3. Diseño por capas

Capa	Componentes	Responsabilidad	Valor del diseño
Presentación	<code>web/J48Controller</code>	Rutas HTTP, validación de cuerpo y anotaciones OpenAPI.	Expone contratos claros sin lógica de modelo.
Dominio	<code>core/J48ModelService</code>	Entrena, carga y ejecuta el clasificador.	Centraliza reglas del ciclo de vida del modelo.
Explicación	<code>ClinicalRiskExplanationService</code>	Sanitiza variables y coordina IA/fallback.	Evita mezclar inferencia determinista con generación de texto.
Persistencia	<code>repository/J48ModelRepository</code>	Ubicaciones y serialización del modelo.	Representa la persistencia real sin introducir una base de datos innecesaria.
Contratos	<code>dto/*</code>	Solicitudes, respuestas de predicción, entrenamiento y error.	Reduce acoplamiento y documenta la API.
Configuración	<code>config/*</code> , YAML	Propiedades tipadas, CORS y OpenAPI.	Externaliza aspectos de entorno y secretos.
Seguridad	<code>security/*</code>	Filtro del token administrativo y cadena de seguridad.	Restringe la operación de mayor impacto.
Errores	<code>exception/*</code>	Mapeo centralizado a HTTP.	Entrega errores consistentes y observables.

Evaluación del diseño

FORTALEZA

Persistencia proporcional

Un repositorio de archivos es apropiado porque el servicio no administra historiales clínicos; conserva un artefacto de modelo y su estructura.

FORTALEZA

Fallos expresivos

Las excepciones de validación, modelo no disponible y proveedor de IA se traducen a códigos HTTP específicos.

ATENCIÓN

Contrato guiado por ARFF

Los nombres y dominios de atributos provienen del conjunto de entrenamiento. Deben publicarse ejemplos y validarse aguas arriba.

ATENCIÓN

Concurrencia de modelo

Los campos son `volatile` y el entrenamiento es sincronizado, una decisión correcta en una instancia; el reentrenamiento multiinstancia requiere coordinación externa.

4. Contratos de API

La documentación interactiva está prevista en `/swagger-ui.html` y la especificación se publica en `/v3/api-docs`. Los endpoints HTTP observados son los siguientes:

Método y ruta	Propósito	Protección	Respuestas relevantes
GET <code>/health</code>	Sonda de disponibilidad del proceso.	Pública	200
GET <code>/info</code>	Estado, atributos y clases del modelo cargado.	Pública	200
POST <code>/predict</code>	Clasifica el riesgo a partir de características.	Pública / red interna	200, 400, 503
POST <code>/predict/explanation</code>	Predicción más explicación generada o de respaldo.	Pública / red interna	200, 400, 502, 503
POST <code>/train</code>	Reentrena y guarda el modelo.	Token si se configura; obligatorio en producción	200, 401, 403, 500

Ejemplo de predicción

```
POST /predict
Content-Type: application/json

{
  "features": {
    "gender": "F",
    "age_group": "adult",
    "sentiment": "neutral",
    "wellbeing": 6.8,
    "anxiety": 3.0,
    "depression": 2.0,
    "attendance": 0.92,
    "days_since_last": 14
  }
}

200 OK
{ "classLabel": "...", "classIndex": 0, "probabilities": { "...": 0.0 } }
```

Los valores y etiquetas exactas deben corresponder con el ARFF desplegado. El ejemplo ilustra la forma del contrato, no una taxonomía clínica prescriptiva.

Errores normalizados

Situación	HTTP	Tratamiento
Cuerpo inválido o vacío	400	DTO de error con detalles de validación cuando aplican.
Modelo aún no cargado	503	Señala indisponibilidad temporal; el consumidor puede reintentar de forma controlada.
Fallo del proveedor IA	502	Se distingue del fallo de predicción para facilitar diagnóstico.

Situación	HTTP	Tratamiento
ARFF/modelo no disponible o error Weka	500	Requiere revisión operativa y del artefacto.

5. Modelo J48 y ciclo de vida

J48 es la implementación de Weka de C4.5: construye un árbol de decisión a partir de atributos y una clase objetivo. Es una elección explicable para clasificación tabular: el árbol permite auditar reglas, aunque la API actual devuelve clase y probabilidades, no el recorrido de la regla.

Entrenamiento y persistencia

- Lee el ARFF mediante `DataSource.read`.
- Define la clase como el último atributo cuando la fuente no la tiene definida.
- Entrena una instancia de `J48`.
- Conserva una cabecera sin filas para construir instancias futuras con el mismo esquema.
- Serializa clasificador y cabecera juntos con `SerializationHelper`.

Controles recomendados sobre el modelo

Control	Estado actual	Mejora propuesta
Versionado de dataset/modelo	PARCIAL	Guardar versión, hash del ARFF, fecha, métricas y responsable en metadatos inmutables.
Validación previa a entrenamiento	BÁSICA	Verificar columnas, tipos, distribución de clase, nulos y tamaño mínimo antes de sustituir el modelo vigente.
Evaluación reproducible	PENDIENTE	Incorporar validación cruzada, matriz de confusión, sensibilidad/especificidad y umbrales de aceptación.
Rollback	PENDIENTE	Conservar versiones anteriores y promover un nuevo modelo de forma atómica tras superar evaluación.
Explicabilidad	BUENA BASE	Publicar árbol/regla por predicción sin revelar datos y con aviso de uso clínico.

Precaución clínica: las probabilidades del árbol no sustituyen una calibración validada. Antes de usar resultados en un flujo asistencial, debe realizarse validación con población representativa, evaluación de sesgos y gobernanza por profesionales responsables.

6. Configuración y despliegue

La configuración se externaliza por variables de entorno. `application.yml` contiene valores por defecto seguros para la integración IA; `application-prod.yml` eleva el requisito del token administrativo.

Variable	Función	Recomendación de producción
SERVER_PORT	Puerto del servicio.	Exponer solo mediante gateway o red privada cuando sea posible.
J48_ARFF_PATH	Fuente de entrenamiento.	Montaje de solo lectura, versionado y control de integridad.
J48_MODEL_PATH	Destino del modelo serializado.	Volumen persistente con permisos mínimos y backup.
J48_AUTO_TRAIN	Entrena al no encontrar modelo.	Desactivarlo en producción salvo procedimiento controlado de bootstrap.
J48_ADMIN_TOKEN	Secreto para <code>/train</code> .	Gestionarlo en un vault/secret manager; rotarlo y no registrarlo.
J48_REQUIRE_ADMIN_TOKEN	Exige token aun vacío.	Mantener <code>true</code> .
J48_CORS_ALLOWED_ORIGINS	Orígenes CORS directos.	Lista explícita; preferir tráfico a través del gateway.
SPRING_AI_OPENAI_ENABLED / OPENAI_API_KEY	Activa proveedor de explicación.	Habilitar solo tras evaluación de privacidad y contrato de tratamiento de datos.

Contenedor

El `Dockerfile` usa construcción multi-stage con Maven y runtime Java 25 JRE. Crea y ejecuta con un usuario no privilegiado, expone el puerto 8080 y define una comprobación de salud contra `/actuator/health`. En Compose, el perfil `j48-java` monta el ARFF como solo lectura y un volumen para `/models`.

Operación mínima

1. Montar dataset validado y volumen persistente para modelos.
2. Configurar token, CORS y secretos en el entorno de despliegue, nunca en el repositorio.
3. Verificar `/actuator/health` y `/info` tras iniciar.
4. Registrar toda acción de entrenamiento fuera del servicio (identidad, versión y aprobación).

7. Seguridad, privacidad y cumplimiento

Área	Control existente	Evaluación
Reentrenamiento	Filtro <code>J48AdminTokenFilter</code> para <code>POST /train</code> .	Adecuado como control básico Debe respaldarse con secreto robusto, TLS y registro de auditoría.
Superficie HTTP	Spring Security permite rutas y añade filtro específico; Actuador expone solo salud e información.	Adecuado para red interna Falta autenticación de servicio a servicio para rutas de inferencia.
CORS	Configuración de orígenes opcional.	Configurable En producción debe ser restrictiva o inexistente tras gateway.
Datos a IA	Lista permitida de ocho variables desidentificadas; no se reenvían identificadores ni notas libres.	Buen principio de minimización Verificar semántica de cada variable y acuerdo con proveedor.
Secretos	Clave IA y token se leen del entorno.	Correcto Falta, como mejora operativa, integrar gestor de secretos y rotación.
Uso clínico	Prompt limita diagnóstico/prescripción y solicita lenguaje profesional conciso.	Control de producto No sustituye validación humana ni evaluación de impacto clínico.

Recomendaciones de privacidad

- Enviar al proveedor externo únicamente los campos estrictamente necesarios y documentar su finalidad.
- No incluir identificadores, texto clínico libre, fechas exactas ni combinaciones que permitan reidentificación.
- Definir retención, trazabilidad y tratamiento de datos con el proveedor de IA antes de habilitarlo.
- Mostrar que el resultado es apoyo a la decisión y registrar el profesional que lo revisa en el sistema clínico principal.

8. Calidad, pruebas y observabilidad

Pruebas existentes

El módulo contiene pruebas de la explicación clínica, filtro de token administrativo y controlador HTTP. También dispone de un ARFF de prueba. La cobertura declarada incluye predicción con el contrato actual, cuerpo vacío, fallback de IA sin filtración de campos no permitidos, restricciones de `/train` y metadatos del modelo.

Dimensión	Estado	Próximo paso concreto
Pruebas unitarias/web	PRESENTES	Ejecutarlas y publicarlas en CI con reporte de cobertura.
Pruebas de integración	PARCIALES	Probar volumen de modelos, arranque sin ARFF, reentrenamiento y recuperación.
Pruebas de carga	PENDIENTES	Medir latencia P95, consumo de memoria y comportamiento durante reentrenamiento.
Observabilidad	BÁSICA	Añadir métricas de predicciones, errores, versión de modelo y duración de entrenamiento.
Calidad de datos/modelo	PENDIENTE	Automatizar evaluación y revisión de sesgos antes de promoción.

Indicadores sugeridos

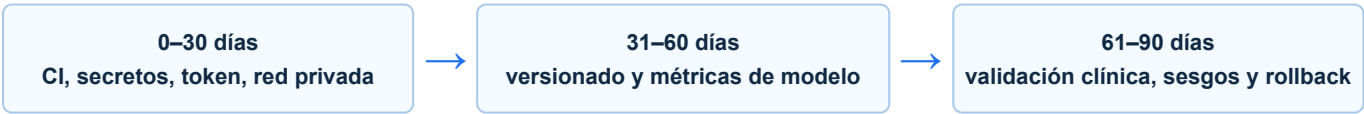
Servicio Disponibilidad, tasa 4xx/5xx, P50/P95 de <code>/predict</code> , memoria JVM y estado del modelo.	Modelo Versión, hash del ARFF, fecha de entrenamiento, distribución de clases y porcentaje de entradas con atributos faltantes.
Seguridad Intentos fallidos de <code>/train</code> , rotación de token, llamadas a la explicación externa y origen de red.	Gobernanza Métricas de desempeño por subgrupo, tasa de revisión humana y eventos de rollback.

La revisión fue estática: en este entorno no se verificó la ejecución de `mvn test`, una llamada a proveedor IA ni un despliegue Docker. El documento no afirma resultados de ejecución que no fueron observados.

9. Riesgos y recomendaciones priorizadas

Prioridad	Riesgo / oportunidad	Acción recomendada	Resultado esperado
ALTA	Reentrenamiento o autoentrenamiento no gobernado.	Exigir token, desactivar autoentrenamiento en producción y promover modelos tras evaluación versionada.	Evita cambios silenciosos de comportamiento clínico.
ALTA	Ausencia de evidencia automatizada de pruebas del módulo.	Incorporar <code>mvn test</code> en CI y bloquear promoción ante fallo.	Reduce regresiones de contrato y seguridad.
ALTA	Uso clínico sin validación de desempeño y sesgo.	Definir protocolo de evaluación, métricas por subgrupo y aprobación clínica antes de uso asistencial.	Uso más seguro y trazable de la predicción.
MEDIA	Rutas de predicción abiertas dentro de la cadena de seguridad.	Aplicar autenticación de servicio a servicio en gateway/red privada; limitar exposición pública.	Menor superficie de abuso y acceso indebido.
MEDIA	Modelo de archivos sin gestión de versiones ni rollback.	Usar almacenamiento versionado, publicación atómica y manifiesto de metadatos.	Reproducibilidad y recuperación rápida.
MEDIA	Campos omitidos degradan inferencia de forma silenciosa.	Publicar esquema del ARFF y añadir validación de dominio/alertas de campos faltantes.	Mejor calidad de entrada y explicabilidad operativa.
MEDIA	Explicación externa puede requerir evaluación de privacidad.	Completar DPIA/validación legal aplicable, minimizar datos y mantener fallback local.	Menor exposición de información sensible.
BAJA	El árbol no se expone para auditoría de decisión.	Agregar endpoint/control interno de reglas y versión del árbol para personal autorizado.	Mayor transparencia para auditoría técnica.

Hoja de ruta sugerida



10. Conclusión

El microservicio `j48-service` constituye una implementación técnicamente coherente de Spring Boot para servir un clasificador J48. La aplicación está organizada en capas, mantiene contratos API documentables, persiste el modelo de forma explícita y separa la explicación generativa de la predicción determinista. El endurecimiento del endpoint de entrenamiento y la configuración externa son avances importantes para un entorno productivo.

Para alcanzar una madurez apta para un flujo clínico, el foco debe desplazarse del código del árbol hacia su gobierno: autenticación entre servicios, control de cambios del modelo, evaluación reproducible, monitoreo, privacidad y revisión humana. Ninguna de estas medidas exige modificar la esencia del diseño actual; son extensiones naturales de la estructura ya existente.

Dictamen técnico

Estado actual: apto como microservicio de apoyo técnico, con mejoras requeridas antes de una operación clínica de alto impacto.

La plataforma cuenta con una base de ingeniería sólida. Debe complementar esa base con evidencia automatizada de calidad, gobernanza de modelos y controles operativos de producción.

Evidencia de código revisada

- `services/j48-service/pom.xml` y `Dockerfile`.
- `src/main/resources/application.yml` y `application-prod.yml`.
- `web/J48Controller.java`, `core/J48ModelService.java` y `core/ClinicalRiskExplanationService.java`.
- `repository/J48ModelRepository.java`, `security/*`, `exception/GlobalExceptionHandler.java`.
- Pruebas bajo `src/test`, configuración de `compose.yaml` y documentación de migración del repositorio.

Documento elaborado a partir de una revisión estática del repositorio disponible. No reemplaza auditoría de seguridad, validación clínica, evaluación regulatoria ni certificación de software médico.